The Journal of Open Source Software

ParcelFlow: A Minimal Data-Driven Workflow Engine for Teaching and Prototyping

Dariusz Młodnicki [1]

1 Technology Department, Key Search AG, Switzerland

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 05 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

ParcelFlow is a **reference implementation** of a data-driven workflow execution engine designed for teaching and research. The 547-line Python implementation demonstrates automatic array processing through pattern matching—nodes declare required data (e.g., `requires=["user"]`) and automatically execute once per array element (`user[0]`, `user[1]`, etc.) without explicit loops.

The implementation prioritizes **conceptual clarity over production features**. Rather than providing production-ready workflow orchestration, ParcelFlow serves as: 1. A teaching tool for workflow concepts (entire engine readable in hours) 2. A research platform for experimenting with execution models (zero dependencies = easy to modify) 3. A reference for understanding data-driven execution semantics

A separate production deployment (TypeScript, currently undergoing case study evaluation at Key Search AG) validates that the paradigm scales beyond the reference implementation, but the Python version published here is explicitly designed for educational and research purposes.

Statement of Need

The Problem

Existing workflow systems fall into two categories, both with drawbacks for specific use cases:

1. **Heavyweight orchestrators** (Airflow (Apache Software Foundation, 2024a), Prefect (Prefect Technologies, 2024)) provide production-grade features but require explicit dependency graphs, significant operational overhead, and weeks of learning curve. Array processing requires either dynamic DAG generation or explicit loops within nodes, mixing orchestration concerns with business logic. Their complexity makes them impractical for teaching workflow concepts or rapid prototyping.

2. **End-user automation platforms** (Zapier, Make) are designed for non-programmers connecting pre-built integrations. While simple for their target use case, they lack the programmability and extensibility developers need for custom data transformations or experimental workflow research.

Neither category serves the need for a **simple, programmable workflow engine** suitable for education and research—one that developers can understand completely, modify easily, and use for prototyping without infrastructure setup.

The Gap ParcelFlow Fills

ParcelFlow provides working, executable software (not a design pattern) for specific use cases:

For educators: - Need a **complete, runnable workflow system** to demonstrate concepts in lectures - Students can execute actual workflows, not pseudocode - Can modify and extend the engine as lab exercises - Example: Assign students to implement custom scheduling algorithms by modifying `workflow_engine.py`

For researchers: - Need a **functioning baseline implementation** to compare alternative execution strategies - Can run experiments with real workflows, not simulations - Zero dependencies enable easy modification and redistribution - Example: Testing new pattern-matching algorithms for nested data structures

For rapid prototyping: - Need **working code** to process data batches (10-100 items) without infrastructure setup - Can prototype data pipelines, test with real data, then decide if production system needed - Example: Processing variable-schema API responses during proof-of-concept phase

Why this is software, not a design pattern: - Complete execution engine with clear termination semantics - Error handling, logging, and debugging support - Command-line interface for running workflows - Example workflows demonstrating real use cases - Can process actual data and produce real results

NOT intended for: - Production systems requiring monitoring, fault tolerance, distributed execution (use Airflow/Prefect) - Workflows with thousands of concurrent tasks - Mission-critical applications requiring high availability

Why Not Just Use Existing Tools?

ParcelFlow addresses a specific niche that existing tools don't serve well:

Existing heavyweight orchestrators (Airflow, Prefect, Nextflow) are designed for production workflows with hundreds of tasks and require significant infrastructure. They excel at scale but have steep learning curves and operational overhead that makes them impractical for teaching or rapid prototyping.

Existing lightweight tools (CWL, Luigi) are simpler but still require understanding DAG construction and scatter/gather declarations. For teaching purposes, students must learn both the tool's syntax and the underlying workflow concepts simultaneously.

ParcelFlow's design choices: - **Minimal implementation** (547 LOC) allows students to read and understand the entire engine - **Zero external dependencies** in reference implementation enables easy modification for research - **Pattern-based array processing** demonstrates scatter/gather without explicit declarations - **Not for production scale** - deliberately trades features for simplicity

The workflow engine's pattern matching and state management add computational overhead compared to direct Python loops, but absolute execution time remains low: processing 100 items completes in 2.3ms on modest hardware (Apple M1). This performance profile is appropriate for the target use case—teaching workflow concepts and prototyping with datasets of 10-100 items where code clarity matters more than execution speed.

Key Features

1. Data-Driven Execution

Nodes declare required data; execution order emerges from availability:

```
class ValidateNode(BaseNode):
    def __init__(self):
        super().__init__(
            requires=["request_data"],     # What I need
```

DRAFT

```
        outputs=["validation_result"]   # What I produce
    )
```

No connections, no predecessors, no DAG definition—only data requirements.

2. Automatic Array Processing

Pattern matching eliminates explicit loops for a specific use case:

```
# Node declares: requires=["user"]
# Engine finds: user[0], user[1], user[2]
# Result: Node runs 3 times automatically—once per index
```

This is similar to scatter/gather in CWL or Nextflow, but implemented purely through data availability checking rather than explicit scatter declarations. For simple array processing workflows, this reduces boilerplate code, though with performance overhead that limits practical use to small datasets.

3. Minimal Dependencies

The reference Python implementation requires only Python 3.8+ with **zero external dependencies**. No pip install, no infrastructure, no configuration files.

This design choice prioritizes: - **Educational accessibility** - Students can run code immediately - **Research portability** - Easy to modify and redistribute - **Conceptual clarity** - No framework abstractions to learn

The production TypeScript implementation (deployed at Key Search AG) adds dependencies for real-world requirements (async execution, persistence, UI), demonstrating how the core paradigm scales when needed.

4. Clear Semantics

Formal execution rules ensure predictable behavior: - State = set of available parcels - Node runs when all required parcels exist - Workflow terminates on success (response parcel) or deadlock (no runnable nodes)

5. Production-Validated

TypeScript implementation deployed at Key Search AG (currently undergoing formal case study evaluation) demonstrates real-world viability: - 50-200 workflows processed daily - 5-20 nodes per workflow - 10-100 array items typical - Async/await for concurrent I/O operations - Case study results pending publication (2025)

Research Enablement

ParcelFlow enables research in several areas:

1. **Alternative execution models**: The separation of data dependencies from execution strategy allows researchers to experiment with different schedulers (sequential, concurrent, distributed) without changing workflow definitions.
2. **Pattern matching algorithms**: The array spreading mechanism opens questions about generalizing to complex data structures (trees, graphs, nested arrays).
3. **Workflow composition**: How can data-driven workflows be composed hierarchically? Can workflows themselves be data?

DRAFT

4. **AI agent integration**: How does the declarative model integrate with agent-based systems that have unpredictable execution patterns?
5. **Formal verification**: The clear semantics enable proving workflow properties like termination, determinism, and data lineage.

The minimal codebase makes ParcelFlow an ideal platform for teaching these concepts and rapid experimentation.

Comparison with Related Work

Scientific workflow systems like Nextflow (Di Tommaso et al., 2017), Snakemake (Köster & Rahmann, 2012), CWL (Amstutz et al., 2016), Swift/T (Wozniak et al., 2013), and Pegasus (Deelman et al., 2015) provide scatter/gather for array processing but require explicit declarations. ParcelFlow's pattern matching is implicit and automatic.

DAG orchestrators like Airflow (Apache Software Foundation, 2024a) and Prefect (Prefect Technologies, 2024) require explicit dependency graphs. ParcelFlow lets structure emerge from data availability, reducing cognitive load for simple workflows.

Dataflow systems like Apache Beam (Apache Software Foundation, 2024b) target large-scale distributed processing. ParcelFlow prioritizes simplicity for SME-scale workflows (10-100 items) over massive parallelism (TB+ datasets).

Reactive streams like RxJS (ReactiveX, 2024) use push-based observable streams for continuous data. ParcelFlow uses pull-based discrete steps for batch-oriented tasks.

Usage Examples

ParcelFlow is executable software with practical applications. Concrete examples demonstrating its implementation:

Basic Array Processing

```python
from workflow_engine import WorkflowEngine
from nodes import ArraySpreadNode, ProcessItemNode, CollectNode

nodes = [
    ArraySpreadNode("spread", "input", "item"),
    ProcessItemNode("process", "item", "processed"),
    CollectNode("collect", "processed", "result", "item_meta")
]

engine = WorkflowEngine()
result = engine.execute_workflow(nodes, {"input": ["a", "b", "c"]})
# Result: {"result": ["PROCESSED: A", "PROCESSED: B", "PROCESSED: C"]}
```

Educational Use

Students implement custom nodes as lab exercises:

```python
class MyCustomNode(BaseNode):
    def __init__(self):
        super().__init__(requires=["data"], outputs=["result"])

    def run(self, parcels, index=None):
```

DRAFT

```
    # Student implements their logic here
    return {"result": transform(parcels["data"].value)}
```

The entire engine is simple enough to understand, debug, and extend—unlike heavyweight systems where the orchestration layer is opaque.

Limitations and Future Work

Current limitations: - Single-threaded Python (reference implementation only) - In-memory parcel storage (not suitable for GB+ datasets) - Flat array processing (nested structures require manual handling) - No native conditional branching or arbitrary loops - Performance optimized for clarity over speed (suitable for 10-100 items; larger datasets should use production orchestrators)

Future work: - Concurrent Python variant using asyncio - Enhanced pattern matching for nested data structures - Visual workflow designer (Python port of TypeScript version) - Conditional execution support - Distributed state management for scaled deployments

Acknowledgements

We thank the early adopters who provided feedback on the system design and validated the production deployment at Key Search AG.

References

Amstutz, P., Crusoe, M. R., Tjináló, N., Chapman, B., Chilton, J., Heuer, M., Kartashov, A., Leehr, D., Ménager, H., Nedeljkovich, M., & others. (2016). *Common workflow language, v1.0*. Figshare. https://doi.org/10.6084/m9.figshare.3115156.v2

Apache Software Foundation. (2024a). *Apache airflow: A platform to programmatically author, schedule and monitor workflows*. https://airflow.apache.org/

Apache Software Foundation. (2024b). *Apache beam: An advanced unified programming model for batch and streaming data processing*. https://beam.apache.org/

Deelman, E., Vahi, K., Juve, G., Rynge, M., Callaghan, S., Maechling, P. J., Mayani, R., Chen, W., Da Silva, R. F., Livny, M., & others. (2015). Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, *46*, 17–35. https://doi.org/10.1016/j.future.2014.10.008

Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, *35*(4), 316–319. https://doi.org/10.1038/nbt.3820

Köster, J., & Rahmann, S. (2012). Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, *28*(19), 2520–2522. https://doi.org/10.1093/bioinformatics/bts480

Prefect Technologies. (2024). *Prefect: Modern workflow orchestration for data and ML engineers*. https://www.prefect.io/

ReactiveX. (2024). *RxJS: Reactive extensions library for JavaScript*. https://rxjs.dev/

Wozniak, J. M., Armstrong, T. G., Wilde, M., Katz, D. S., Lusk, E., & Foster, I. T. (2013). Swift/t: Large-scale application composition via distributed-memory dataflow processing. *2013 13th IEEE/ACM International Symposon Cluster, Cloud, and Grid Computing*, 95–102. https://doi.org/10.1109/CCGrid.2013.99

DRAFT